

SOFTWARE ENGINEERING MINI PROJECT

**RETAIL INVENTORY AND STOCK MANAGEMENT
SYSTEM**

Software Architecture & Design Specification

Deliverable Part-1 | Team E10

Team Members

Name	SRN
Malavika A	PES1UG24CS257
Manoj Kumar	PES1UG24CS264
Mayank M	PES1UG24CS266
LG Vivyn Krishnaa	PES1UG24CS248

1. Introduction

1.1 Purpose

This document describes the software architecture and detailed design of the **Retail Inventory and Stock Management System**. It explains how the requirements captured in the Software Requirements Specification are realised as a set of components, interfaces, data structures and interactions, and it provides enough detail for the team to implement the system and for the Test Plan to be traced back to design elements. It translates the Phase 1 requirements into components, interfaces, data formats and behaviour that the team will implement in C++ and verify through the Test Plan.

1.2 Scope

RISMS is a menu-driven C++ console application that lets the Administrator and Inventory Staff maintain product records, search and view products, increase or decrease stock, check availability, review stock movement history and view an inventory summary with low-stock items. All data is persisted in local files. A graphical or web interface, a database server, barcode scanners, point-of-sale integration and external retail-system integration are outside the scope of Part-1. The two actor roles and their permissions are defined in the SRS.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
RISMS	Retail Inventory and Stock Management System
SRS	Software Requirements Specification (Phase 1 document)
FR / VFR / NFR	Functional / Verifiable Functional / Non-Functional Requirement
SEC	Security requirement (SEC-01–SEC-04 in the SRS)
UC	Use Case (UC01–UC11 as in the SRS)
Stock-in / Stock-out	An adjustment that increases / decreases a product's quantity
Ledger	The append-only stock movement history file (stock_ledger.csv)
Low-stock threshold	Configured quantity at or below which a product is flagged (VFR09)
Atomic write	Writing to a temporary file and renaming it over the original, so a crash never leaves a half-written file
Repository	Class that hides file formats and I/O from the business logic

1.4 References

- SRS – Retail Inventory and Stock Management System, Team E10, Phase 1 deliverable (FR01–FR14, VFR01–VFR09, UC01–UC11).
- Test Plan – Retail Inventory and Stock Management System, Team E10, Part-1 deliverable.
- IEEE Std 1016-2009, Systems Design – Software Design Descriptions.
- ISO/IEC/IEEE 42010:2011, Architecture description.
- SEI CERT C++ Coding Standard .

1.5 Document Overview

Section 2 presents the architecture: drivers, chosen pattern, component diagram and descriptions, deployment, data design, traceability to requirements and the security architecture. Section 3 presents the detailed design: UML sequence diagrams, API (module interface) design, error handling and key algorithms. Section 4 lists constraints and assumptions and Section 5 records the design-to-test hooks used by the Test Plan.

1.6 SRS Non-Functional and Security Requirement Basis

The SRS defines NFR01–NFR08 and security requirements SEC-01–SEC-04. The design decisions in this document are intended to realize those requirements. Additional implementation safeguards, such as salted password hashing, login lockout and owner-only data-directory permissions, are design choices; they are not separate requirement IDs in the SRS.

ID	SRS requirement / design basis
NFR01 Usability	Provide a simple menu-driven console interface with clear prompts and messages for inventory operations.
NFR02 Input Validation	Provide appropriate validation messages for invalid or incomplete product or stock information.
NFR03 Reliability	Maintain accurate product and stock information during inventory operations.
NFR04 Data Persistence	Preserve product records and stock movement history in local files after the application is closed and restarted.
NFR05 Maintainability	Organize the system into product management, stock adjustment, inventory lookup, and history/report modules to support implementation and testing.
NFR06 Portability	Implement using C or C++ and standard file I/O on a compatible computer environment.
NFR07 Performance	Perform basic inventory operations without unnecessary processing delays under normal usage; the SRS specifies no numeric timing threshold.
NFR08 Local Operation	Operate locally without dependence on an external database, hosted storage, or network communication.
SEC-01	Allow inventory functions to be accessed only by authorized users.
SEC-02	Restrict unauthorized modification or removal of product records.
SEC-03	Validate stock adjustments before updating stored stock quantities.
SEC-04	Maintain the integrity of stock movement history when successful stock adjustments are recorded.

Stakeholders and Concerns

Stakeholder	Primary design concerns
Administrator / Inventory Staff	Simple menus, clear messages, accurate stock figures, no lost data
Store owner	Correct low-stock reporting, audit trail of stock changes, restricted access
Developers	Modular code, clear interfaces, independent module testing
Testers / evaluators	Every requirement traceable to a component and a test case

2. Architectural Design

2.1 Architectural Drivers

The following requirements influence the architecture the most:

- **Correct stock arithmetic (VFR05, VFR06):** quantity must change only through controlled stock-in and stock-out operations, never through a general product edit.
- **Traceable history (FR09, VFR07, additional audit fields):** every successful adjustment must reach the ledger, so product save and ledger append must be coordinated in one place.
- **Persistence without a database (FR13, FR14, VFR08, NFR04):** files must survive restarts and crashes, so all file I/O is concentrated in one component that performs atomic writes.
- **Modifiability (NFR05):** the SRS schedule analysis splits the work into product records, stock adjustment, lookup and history/report modules that are developed and tested independently, so module boundaries must be explicit.

2.2 Architectural Pattern

RISMS uses a strict three-layer architecture with the Repository pattern at the data layer. Each layer calls only the layer directly below it. The presentation layer never touches a file, and the service layer never knows a file name or format.

Layer	Responsibility	Modules
Presentation layer	Show menus, read and parse input, print results and error messages, check that a session exists before dispatching a command.	MenuController, ConsoleIO
Business logic layer (services)	Business rules: authentication, validation, stock arithmetic, ledger entries, low-stock detection, reports.	AuthService, ProductService, StockService, HistoryService, ReportService, Validator
Data access layer (repositories)	Load, cache, save and query records; hide file formats; atomic writes and appends.	UserRepository, ProductRepository, MovementRepository, ConfigRepository
Persistent storage	CSV and text files in a local data/ directory.	Local file system

Why this pattern

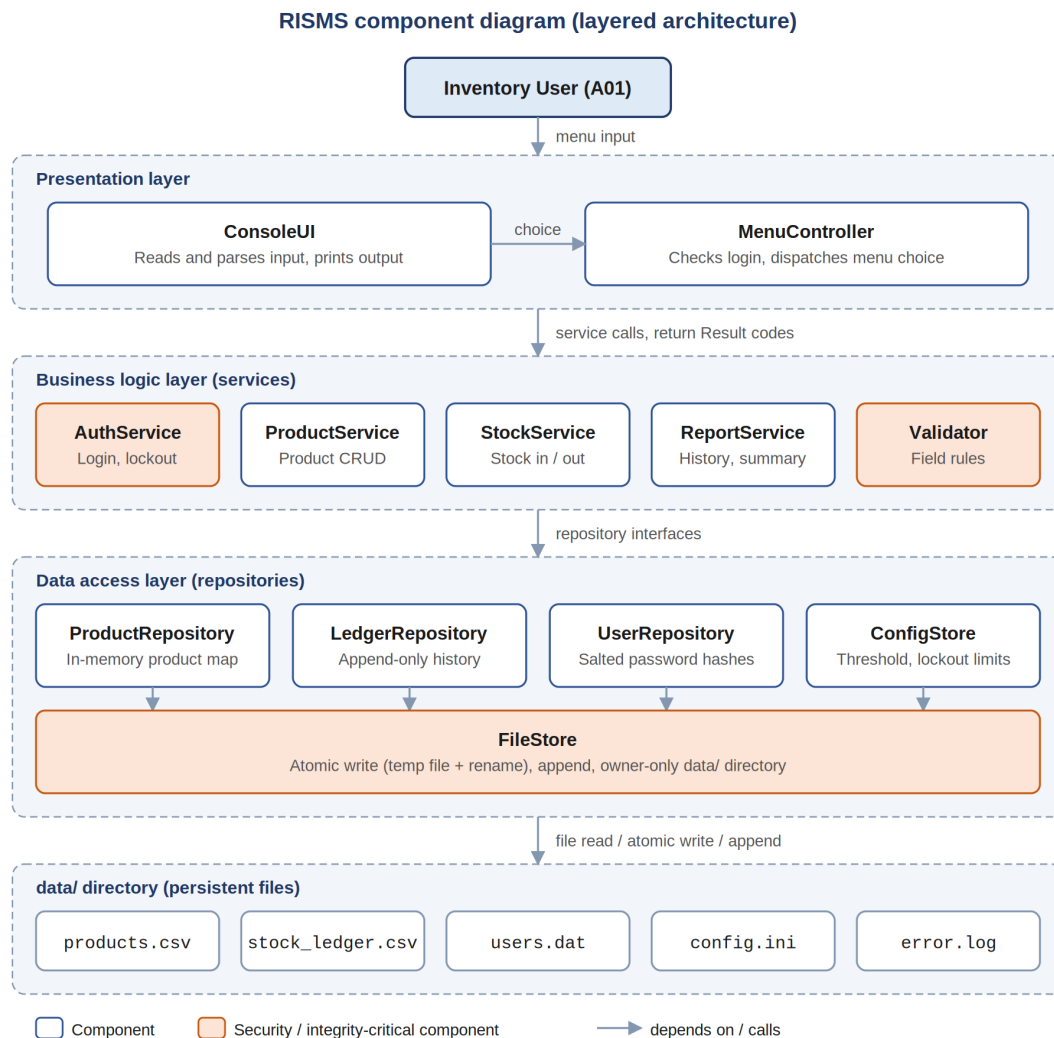
- **Separation of concerns:** the rule “stock-out cannot exceed available quantity” lives only in StockService, so it is tested once without driving the console (NFR05).
- **Testability:** services receive repository interfaces in their constructors, so unit tests pass in-memory fakes instead of real files.
- **Replaceable persistence:** moving from CSV files to SQLite later means one new repository class, not a rewrite of the business logic.
- **Right-sized for the project:** a single process with an in-memory cache needs no threads, sockets or external libraries.

Alternatives considered

Alternative	Reason for rejection
Single-file procedural program (functions and global arrays)	Fastest to start, but validation, file I/O and menu code become entangled, which makes VFR tests hard to isolate.
Model–View–Controller	Adds little over layering for a console application with one view; the controller role is already covered by MenuController.
Client–server with a database	Out of scope per the SRS (file handling, no external database) and adds installation cost.

2.3 Component Diagram

Figure 1 shows the components of RISMS grouped by layer and the dependencies between them. Orange components are security- and integrity-critical. The generic actor shown at the boundary represents both SRS roles (Administrator and Inventory Staff); role-specific authorization is enforced by MenuController.



2.4 Component Descriptions

Component	Responsibility	Provided interface	Requirements
ConsoleUI	Prints menus, tables and messages; reads lines with <code>std::getline</code> ; parses numbers safely.	<code>readText()</code> , <code>readInt()</code> , <code>readDouble()</code> , <code>show()</code>	NFR01, NFR02
MenuController	Runs the login screen and main menu loop; blocks every command until a session exists; maps each choice to a service call.	<code>run()</code>	FR01, SEC-01
AuthService	Verifies credentials against salted hashes, counts failures, applies lockout, holds the current user.	<code>login()</code> , <code>logout()</code> , <code>isAuthenticated()</code> , <code>currentUser()</code>	FR01, SEC-01 (additional authentication safeguards: additional authentication safeguards)
ProductService	Add, update, remove, search and list products; rejects duplicate IDs.	<code>addProduct()</code> , <code>updateProduct()</code> , <code>removeProduct()</code> , <code>findById()</code> , <code>searchByName()</code> , <code>listAll()</code>	FR02–FR06, FR11, SEC-02, VFR01–VFR04
StockService	Stock-in and stock-out with arithmetic and availability checks; one ledger entry per successful change.	<code>stockIn()</code> , <code>stockOut()</code> , <code>getQuantity()</code>	FR07–FR09, SEC-03, SEC-04, VFR05–VFR07
ReportService	Stock history per product; inventory summary with totals and low-stock list.	<code>historyFor()</code> , <code>inventorySummary()</code>	FR10, FR12, VFR09
Validator	Single home for field rules: required fields, lengths, allowed characters, non-negative numbers.	<code>validateProduct()</code> , <code>validateQty()</code>	NFR02, SEC-03, VFR02
ProductRepository	Loads <code>products.csv</code> into an <code>unordered_map</code> ; saves the full file on change.	<code>load()</code> , <code>get()</code> , <code>exists()</code> , <code>all()</code> , <code>put()</code> , <code>erase()</code> , <code>save()</code>	FR13, FR14, NFR04, NFR07
LedgerRepository	Appends movements to <code>stock_ledger.csv</code> ; reads history for a product; no update or delete.	<code>append()</code> , <code>forProduct()</code> , <code>nextId()</code>	FR09, FR10, SEC-04 (additional audit fields: additional audit fields)
UserRepository	Reads and writes <code>users.dat</code> (user ID, salt, hash).	<code>find()</code> , <code>create()</code>	FR01, SEC-01 (additional password handling: additional password handling)
ConfigStore	Reads <code>config.ini</code> ; falls back to defaults if a key is missing.	<code>lowStockThreshold()</code> , <code>maxAttempts()</code> , <code>lockoutSeconds()</code>	VFR09 (additional lockout settings: additional lockout settings)
FileStore	All file I/O: creates <code>data/</code> with owner-only permissions, reads lines, atomic rewrite, append.	<code>ensureDataDir()</code> , <code>readLines()</code> , <code>writeAtomic()</code> , <code>appendLine()</code>	FR13, FR14, NFR04 (additional file safeguards: additional file safeguards)

2.5 Deployment View

RISMS is deployed on a single node; there are no servers or network links:

- **Store PC:** Windows 10+ or Linux machine able to run a console program.
- **Executable:** risms (Linux) or risms.exe (Windows), containing all three layers, built with `g++ -std=c++17 -O2 src/*.cpp -o risms` or the equivalent CMake target.
- **Data directory:** data/ next to the executable, created on first run with owner-only permissions, holding products.csv, stock_ledger.csv, users.dat, config.ini and error.log.

2.6 Data Design

The principal persistent entities and their key attributes are shown below. Attributes marked (U) are unique, (I) are indexed in memory.

Entity (file)	Key attributes	Notes
Product (products.csv)	productId (U, I), name, category, unitPrice, quantity	ID 1–10 letters/digits; name 1–50 chars; price and quantity ≥ 0. Whole file rewritten atomically on change.
StockMovement (stock_ledger.csv)	movementId (U), productId, type {IN, OUT}, quantityChange, resultingQty, timestamp, userId, reason	Append-only (additional audit fields). quantityChange > 0; direction given by type. History kept after a product is removed.
User (users.dat)	userId (U), role {ADMIN, STAFF}, saltHex, hashHex	Password never stored in plain text (additional design safeguard). Role is used to enforce SRS permissions. Initial administrator account created on first run.
Config (config.ini)	low_stock_threshold (default 10), max_login_attempts (3), lockout_seconds (60)	Defaults used if a key is missing.
ErrorLog (error.log)	timestamp, level {INFO, WARN, ERROR}, component, message	Security events and I/O errors; never shown to the user.

2.7 Traceability to Requirements

Table 2.7-a maps every functional and verifiable requirement in the SRS to the use case, the responsible component and the operation that realises it. Table 2.7-b does the same for non-functional and security requirements.

Table 2.7-a: Functional requirements

Requirement	Use case	Component(s)	Operation / design element
FR01	UC01	MenuController, AuthService	AuthService::login(); session check before every command
FR02, VFR01, VFR02	UC02	ProductService, Validator	addProduct() → DUPLICATE_ID / INVALID_INPUT
FR03	UC03	ProductService, Validator	updateProduct() (quantity not editable here)
FR04	UC04	ProductService	removeProduct() with Y/N confirmation

Requirement	Use case	Component(s)	Operation / design element
FR05, VFR03, VFR04	UC05	ProductService	findById(), searchByName() → NOT_FOUND
FR06	UC06	ProductService, ConsoleUI	findById() and detail view
FR07, VFR05, VFR06	UC07	StockService	stockIn(), stockOut() → INSUFFICIENT_STOCK
FR08	UC08	StockService	getQuantity()
FR09, VFR07	UC07	StockService, LedgerRepository	LedgerRepository::append() after every successful adjustment
FR10	UC09	ReportService	historyFor(), newest first
FR11	UC10	ProductService	listAll(), sorted by product ID
FR12, VFR09	UC11	ReportService, ConfigStore	inventorySummary(): quantity ≤ threshold
FR13	UC02, UC07	Repositories, FileStore	save(), append(), writeAtomic()
FR14, VFR08	UC01	ProductRepository, LedgerRepository	load() at start-up; atomic write guarantees intact files

Table 2.7-b: Non-functional and security requirements

Requirement	Design decision that realizes it
NFR01	ConsoleUI and MenuController provide a menu-driven console, clear prompts, and operation/result messages.
NFR02	ConsoleUI parses input safely; Validator checks required fields, lengths, types, and non-negative values and returns actionable validation messages.
NFR03	ProductService and StockService validate changes; stock arithmetic and persistence handling preserve accurate quantities during operations.
NFR04	ProductRepository and LedgerRepository load and persist product records and movement history in local files across restarts.
NFR05	Three-layer architecture separates presentation, services, and repositories; service/repository interfaces support independent testing.
NFR06	Implementation uses C/C++ and standard file input/output on a compatible local computer.
NFR07	Product records are cached in an in-memory map for lookup; basic operations are designed for normal local use. No numeric service-level target is added.
NFR08	Deployment is a single local node using local files; no external database, hosted storage, or network service is required.
SEC-01	AuthService and MenuController require authenticated access before protected inventory functions are dispatched.
SEC-02	ProductService validates and controls product-record modification/removal; role checks restrict UC04 removal to Administrator.
SEC-03	StockService validates adjustment type and quantity, including preventing stock-out beyond available quantity, before updating stock.
SEC-04	LedgerRepository appends successful movements and does not expose update/delete operations; product-save/ledger coordination protects history integrity.

2.8 Security Architecture

2.8.1 Security objectives

Objective	Description
SO-01 Access control	Ensure only authorized users can access the inventory application and perform inventory operations.
SO-02 Product protection	Protect product records and stock information from unauthorized modification or removal.
SO-03 Stock integrity	Maintain the integrity of stock quantities and stock movement history during inventory operations.
SO-04 Availability / consistency	Ensure stored inventory data remains available and consistent after application restart.

2.8.2 Security controls by layer

Layer	Control	Objective / SRS requirements requirement
Presentation	Input read as whole lines into std::string; numeric parsing in try/catch; no raw file paths or exception text printed.	SO-03 / NFR02
Session	Login gate before the main menu; session cleared on logout and on exit.	SO-01 / SEC-01
Authentication	Salted SHA-256 hashes, constant-time compare, 3-attempt lockout for 60 s, generic "incorrect user ID or password" message.	SO-01 / SEC-01 (hashing and lockout are additional safeguards)
Business logic	Every service re-validates arguments; quantity changed only via stockIn/stockOut; stock-out bounded by available quantity.	SO-02, SO-03 / SEC-02, SEC-03
Data access	Append-only ledger; atomic rewrite of products and users; malformed lines skipped and logged, never guessed.	SO-03, SO-04 / SEC-04, NFR04
Storage	data/ directory restricted to the owning OS account.	SO-01 / SEC-01 (owner-only file permissions are an additional safeguard)
Audit	error.log records login success, failure, lockout, logout and I/O errors.	SO-03 / SEC-04

2.8.3 Access control matrix

The SRS defines two actors: Administrator and Inventory Staff. Both must authenticate to use inventory functions; the controller also checks the actor's role so that UC04 (Remove Product) and UC11 (View Inventory Summary) remain Administrator-only.

Operation	Administrator	Inventory Staff	Unauthenticated
Login / Exit	Yes	Yes	Yes
Add product (UC02)	Yes	Yes	No
Update product (UC03)	Yes	Yes	No
Remove product (UC04)	Yes	No	No
Search product (UC05)	Yes	Yes	No
View product details (UC06)	Yes	Yes	No
Adjust stock (UC07)	No	Yes	No

Operation	Administrator	Inventory Staff	Unauthenticated
Check stock availability (UC08)	Yes	Yes	No
View stock movement history (UC09)	No	Yes	No
View inventory (UC10)	Yes	Yes	No
View inventory summary (UC11)	Yes	No	No
Load saved inventory (UC01)	Yes	Yes	No

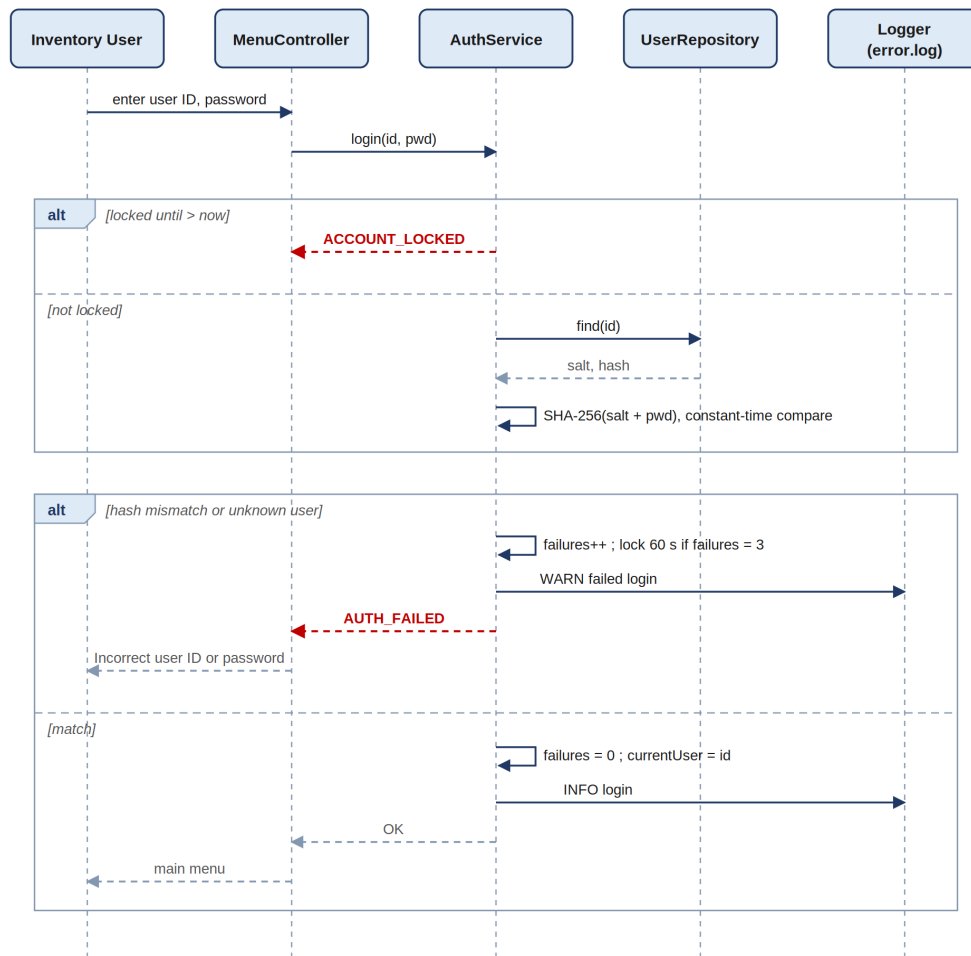
3. Detailed Design

3.1 UML Sequence Diagrams

Three interaction scenarios are modelled: **SD1 – User Login (UC01)**, **SD2 – Add Product (UC02)** and **SD3 – Stock-out with Ledger Entry (UC07)**. Solid arrows are calls, dashed arrows are returns and red arrows are error returns.

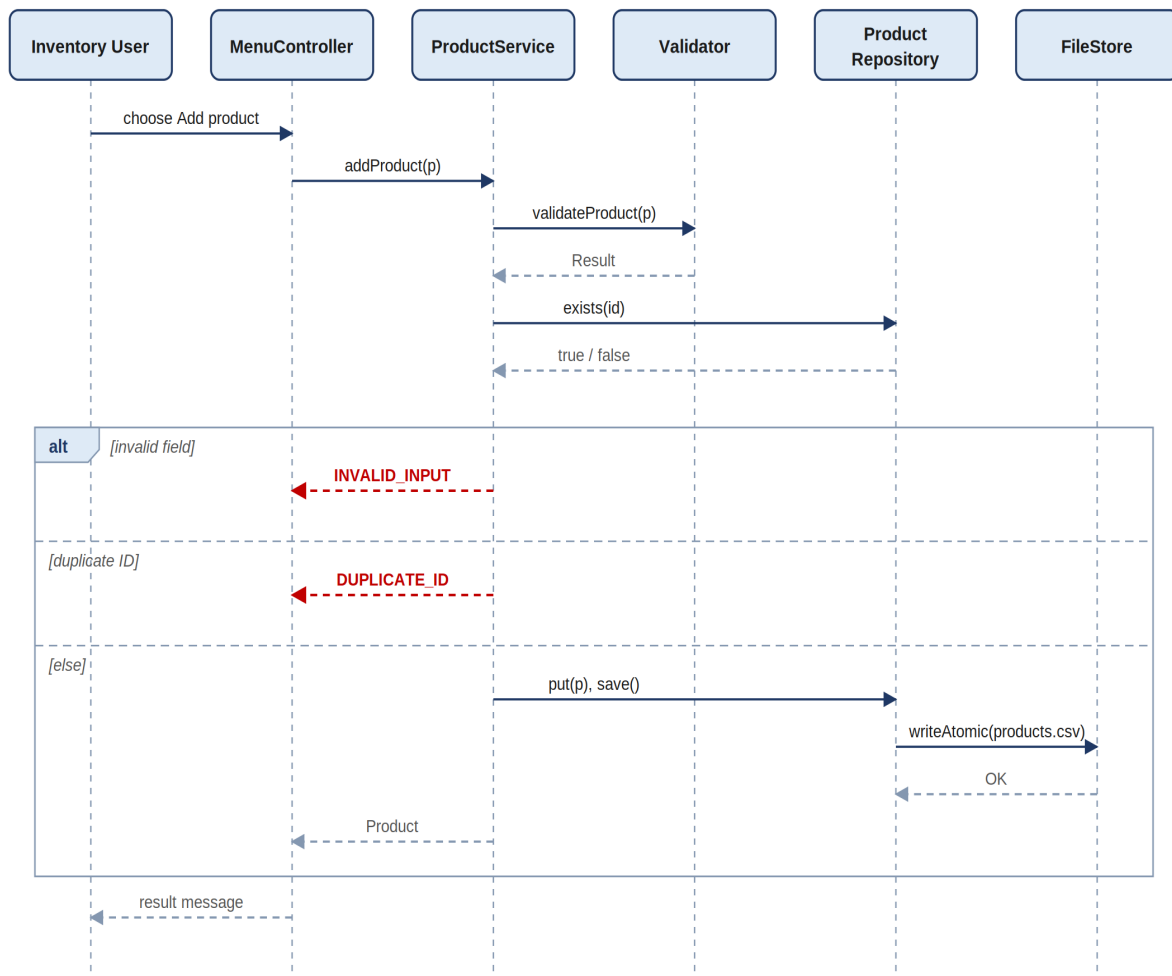
3.1.1 SD1 – User Login (UC01)

SD1 - User login with lockout (UC01)



3.1.2 SD2 – Add Product (UC02)

SD2 - Add product (UC02)



Validation runs first, then the duplicate check against the in-memory map. **INVALID_INPUT** (with the field name) takes precedence over **DUPLICATE_ID**. Only a valid, new product reaches `save()`.

3.1.3 SD3 – Stock-out with Ledger Entry (UC07)

SD3 - Stock-out with ledger entry (UC07)

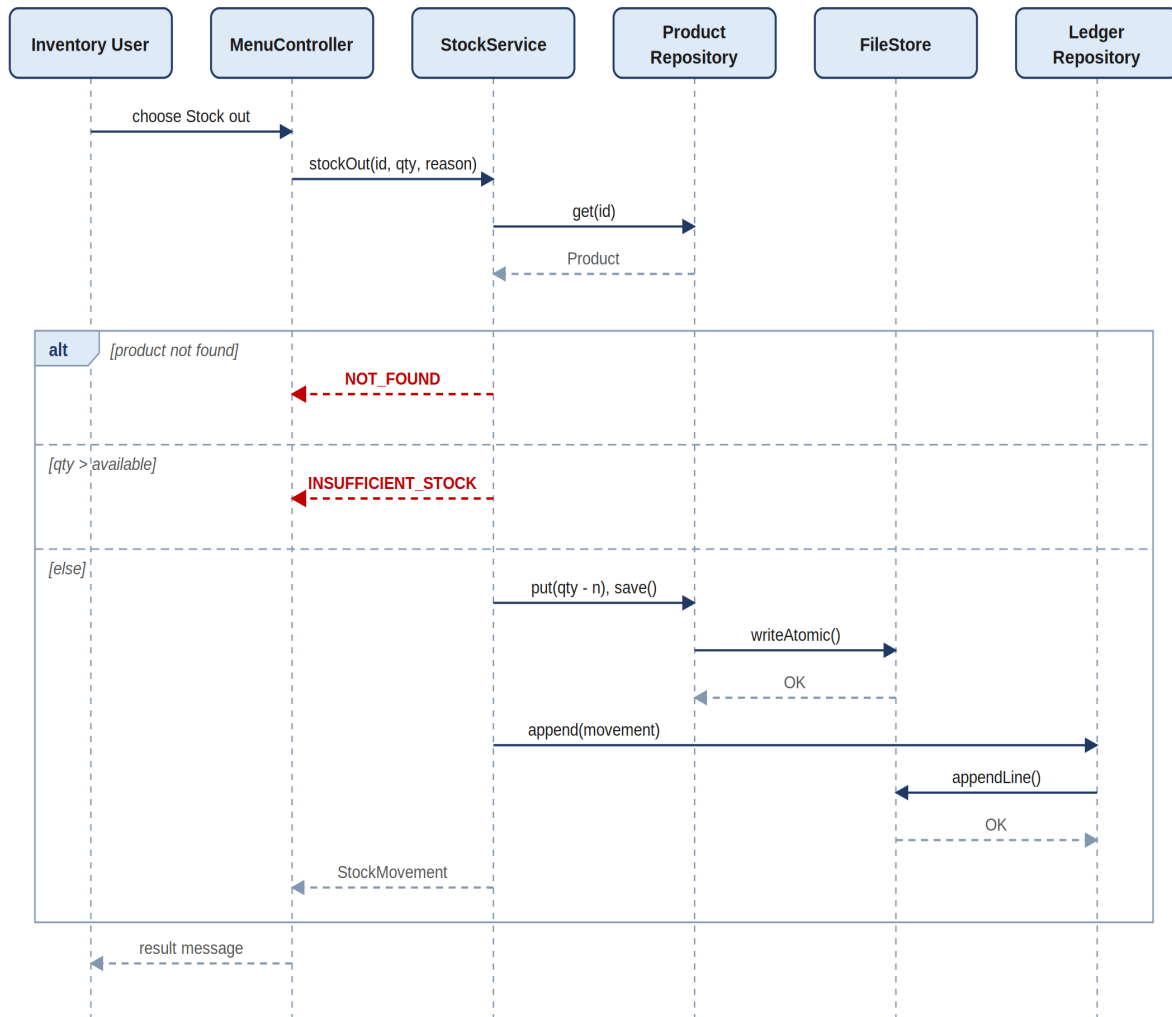


Figure 4: Sequence diagram for stock-out and ledger logging (FR07, FR09, VFR06, VFR07)

The product file is rewritten atomically before the movement is appended. If the append fails, the quantity is restored and the product file re-saved (Section 3.3.1).

3.2 API Design

RISMS has no network API. Its API is the set of C++ class interfaces between layers, plus the console menu that users interact with.

3.2.1 Conventions

- Every service operation returns **Result<T>**, carrying either a value or an ErrorCode with a human-readable message; expected failures never throw.
- Identifiers are std::string; quantities are int; prices are double rounded to two decimals; timestamps are ISO-8601 local time from the system clock.
- Services validate all arguments themselves, even when ConsoleUI has already checked them.

- Quantity is changed only through stockIn()/stockOut(), never through updateProduct(), so every change reaches the ledger.

3.2.2 Service operation summary

Operation	Purpose	Returns	Main errors
AuthService::login(id, pwd)	Authenticate and open a session	Result<void>	AUTH_FAILED, ACCOUNT_LOCKED
AuthService::logout()	End session, log event	void	—
ProductService::addProduct(p)	Create product record	Result<Product>	INVALID_INPUT, DUPLICATE_ID, FILE_IO_ERROR
ProductService::updateProduct(id, name, category, price)	Modify name, category or price	Result<Product>	NOT_FOUND, INVALID_INPUT, FILE_IO_ERROR
ProductService::removeProduct(id)	Remove product (history kept)	Result<void>	NOT_FOUND, FILE_IO_ERROR
ProductService::findById(id)	Exact lookup	Result<Product>	NOT_FOUND
ProductService::searchByName(text)	Case-insensitive substring search	Result<vector<Product>>	NOT_FOUND
ProductService::listAll()	All products sorted by ID	vector<Product>	—
StockService::stockIn(id, qty, reason)	Increase quantity, log movement	Result<StockMovement>	NOT_FOUND, INVALID_INPUT, FILE_IO_ERROR
StockService::stockOut(id, qty, reason)	Decrease quantity, log movement	Result<StockMovement>	NOT_FOUND, INVALID_INPUT, INSUFFICIENT_STOCK
StockService::getQuantity(id)	Current available quantity	Result<int>	NOT_FOUND
ReportService::historyFor(id)	Movements for a product, newest first	Result<vector<StockMovement>>	NOT_FOUND
ReportService::inventorySummary()	Totals and low-stock list	InventorySummary	—

3.2.3 Common types

```
enum class ErrorCode { OK, INVALID_INPUT, DUPLICATE_ID, NOT_FOUND,
    INSUFFICIENT_STOCK,
    AUTH_FAILED, ACCOUNT_LOCKED, NOT_AUTHENTICATED,
    FILE_IO_ERROR, DATA_CORRUPT };

template <typename T> struct Result {
    ErrorCode code = ErrorCode::OK;
    std::string message;           // shown to the user by ConsoleUI
    std::optional<T> value;
    bool ok() const { return code == ErrorCode::OK; }
};

enum class AdjustType { IN, OUT };
struct Product { std::string id, name, category; double unitPrice; int quantity; };
struct StockMovement { long id; std::string productId; AdjustType type;
    int change, resultingQty; std::string timestamp, userId,
    reason; };
struct InventorySummary { int totalProducts; long totalUnits; double totalValue;
    std::vector<Product> lowStock; };
```

3.2.5 Console menu (user-facing interface)

Key	Menu option	Service call	Use case	Authorized actor(s)
1	Add product	addProduct()	UC02	Administrator / Inventory Staff
2	Update product	updateProduct()	UC03	Administrator / Inventory Staff
3	Remove product (asks Y/N)	removeProduct()	UC04	Administrator only
4	Search product (by ID or name)	findByld() / searchByName()	UC05, UC06	Administrator / Inventory Staff
5	Stock in	stockIn()	UC07	Inventory Staff
6	Stock out	stockOut()	UC07	Inventory Staff
7	Check stock	getQuantity()	UC08	Administrator / Inventory Staff
8	Stock history	historyFor()	UC09	Inventory Staff
9	List all products	listAll()	UC10	Administrator / Inventory Staff
10	Inventory summary	inventorySummary()	UC11	Administrator only
0	Log out and exit	logout()	UC01	Authenticated user

3.3 Error Handling

3.3.1 Strategy

- **Validate before mutate:** every service checks all arguments before changing any state (VFR02, NFR02 / SEC-03).
- **Typed error codes:** services return Result with an ErrorCode; they never print or exit themselves.
- **One place for presentation:** ConsoleUI turns each ErrorCode into the message in Section 3.3.3; details go only to error.log.
- **Consistency on failure:** a service updates the in-memory copy, calls save()/append(), and on failure restores the previous value before returning FILE_IO_ERROR. For stock adjustments the product file is saved first, then the ledger entry is appended; if the append fails the quantity is restored and the product file re-saved (VFR07, NFR04, SEC-04).
- **Corrupt data at start-up:** a line that fails to parse is skipped, its line number logged, and the user warned once; loading continues.
- **Unexpected errors:** a top-level try/catch in main() logs the exception and exits cleanly without writing partial data.

3.3.2 Standard error presentation

```
[ERROR] INSUFFICIENT_STOCK
Only 12 units of P0001 are available; cannot remove 20.
Press Enter to return to the menu.
```

```
error.log:
2026-09-28T14:07:40 WARN StockService stockOut P0001 qty=20 available=12 user=staff01
```

3.3.3 Error catalogue

Code	Raised when	User-facing message	Recovery	Related requirement
INVALID_INPUT	Empty field, negative price or quantity, non-numeric or over-length input	Invalid <field>: <reason>.	Re-prompt the field	VFR02, NFR02 / SEC-03
DUPLICATE_ID	Product ID already stored	Product ID <id> already exists.	Ask for another ID	VFR01
NOT_FOUND	No product matches the key	No product found for '<key>'.	Return to menu	VFR04
INSUFFICIENT_STOCK	Stock-out quantity exceeds available	Only <n> units available; cannot remove <m>.	Return to menu, quantity unchanged	VFR06
AUTH_FAILED	Wrong user ID or password	Incorrect user ID or password.	Re-prompt; count attempt	FR01, additional password handling
ACCOUNT_LOCKED	3 consecutive failed logins	Too many attempts. Try again in 60 seconds.	Wait, then re-prompt	additional lockout settings
NOT_AUTHENTICATED	Command without a session	Please log in first.	Show login screen	SEC-01
FILE_IO_ERROR	File cannot be opened, written or renamed	Could not save data. Your last change was not applied.	Roll back, log, return to menu	NFR04, SEC-04
DATA_CORRUPT	A stored line fails to parse at load	Warning: <n> unreadable record(s) skipped (see error.log).	Skip line, keep loading	FR14, VFR08

3.4 Key Algorithms

3.4.1 Stock-out with ledger entry (FR07, FR09, VFR06, VFR07)

```

stockOut(id, qty, reason):
    if qty <= 0: return INVALID_INPUT
    p = products.get(id); if p is none: return NOT_FOUND
    if qty > p.quantity: return INSUFFICIENT_STOCK
    old = p.quantity
    p.quantity = old - qty; products.put(p)
    if products.save() fails: p.quantity = old; products.put(p); return FILE_IO_ERROR
    m = StockMovement(ledger.nextId(), id, OUT, qty, p.quantity, now(),
auth.currentUser(), reason)
    if ledger.append(m) fails:
        p.quantity = old; products.put(p); products.save(); return FILE_IO_ERROR
    return m

```

stockIn() is identical except that it has no availability check and sets new quantity = old + qty (VFR05).

3.4.2 Atomic file write (NFR04, SEC-04)

```

writeAtomic(path, lines):
    tmp = path + ".tmp"
    write all lines to tmp; flush; close
    if any stream error: delete tmp; return FILE_IO_ERROR
    std::filesystem::rename(tmp, path) // replaces the original in one step

```

3.4.3 Login with lockout (FR01, additional password handling, additional lockout settings)

```
login(id, pwd):
    if now() < lockUntil: return ACCOUNT_LOCKED
    u = users.find(id)
    ok = u exists and constantTimeEqual(sha256(u.salt + pwd), u.hash)
    if not ok:
        failures += 1; log WARN
        if failures >= maxAttempts: lockUntil = now() + lockoutSeconds; failures = 0
        return AUTH_FAILED
    failures = 0; current = id; log INFO; return OK
```

4. Design Constraints and Assumptions

- The system is a standalone C++17 console application running on one store PC. It supports the two SRS actor roles (Administrator and Inventory Staff); only one instance runs against a data/ directory at a time.
- Persistence uses files only, as required by the SRS; no external database or network service is used.
- Only free tools are used (g++/MSVC Community, CMake), in line with the economic feasibility in the SRS.
- The SRS defines two actors, Administrator and Inventory Staff. Role-based access shall follow the SRS use cases: UC04 and UC11 are Administrator-only; the remaining use cases are available to the actors specified in the SRS.
- The system clock of the store PC is the single time source for ledger timestamps.
- The low-stock threshold is one store-wide value in config.ini (default 10 units).